

Metaplex

Security Assessment

January 21st, 2025 — Prepared by OtterSec

Tuyết Duong tuyet@osec.io

Robert Chen r@osec.io

Table of Contents

Executive Summary **2**

Overview 2

Key Findings 2

Scope **3**

Findings **4**

Vulnerabilities **5**

OS-MPX-ADV-00 | Discrepancy in Burning NFT with an Escrow Account 6

OS-MPX-ADV-01 | Missing Mint Ownership Validation 7

OS-MPX-ADV-02 | Failure to Decrease Collection Size on NFT Burn 8

OS-MPX-ADV-03 | Absence of Verification for Token Mint 9

General Findings **10**

OS-MPX-SUG-00 | Incorrect Token Standard in Mint Validation 11

OS-MPX-SUG-01 | Code Refactoring 12

Appendices

Vulnerability Rating Scale **13**

Procedure **14**

© 2024 Otter Audits LLC. All Rights Reserved.

01 — Executive Summary

Overview

Metaplex Foundation engaged OtterSec to assess the `mpl-token-metadata` program. This assessment was conducted between December 16th, 2024 and January 10th, 2025. For more information on our auditing methodology, refer to Appendix B.

Key Findings

We produced 6 findings throughout this audit engagement.

In particular, we identified a vulnerability where burning an NFT with a token-owned escrow account permanently locks the escrow account as the token balance check fails, preventing token transfers or account closure (OS-MPX-ADV-00). Additionally, the Utilize instruction fails to update the collection metadata when an NFT with a burn use method is depleted and burned, resulting in incorrect tracking of the collection size (OS-MPX-ADV-02).

We also recommended modifying the codebase for improved functionality and mitigation of security issues (OS-MPX-SUG-01).

© 2024 Otter Audits LLC. All Rights Reserved.

02 — Scope

The source code was delivered to us in a Git repository at https://github.com/metaplex-foundation/mpl-token-metadata. This audit was performed against commit 7c1c905.

A brief description of the program is as follows:

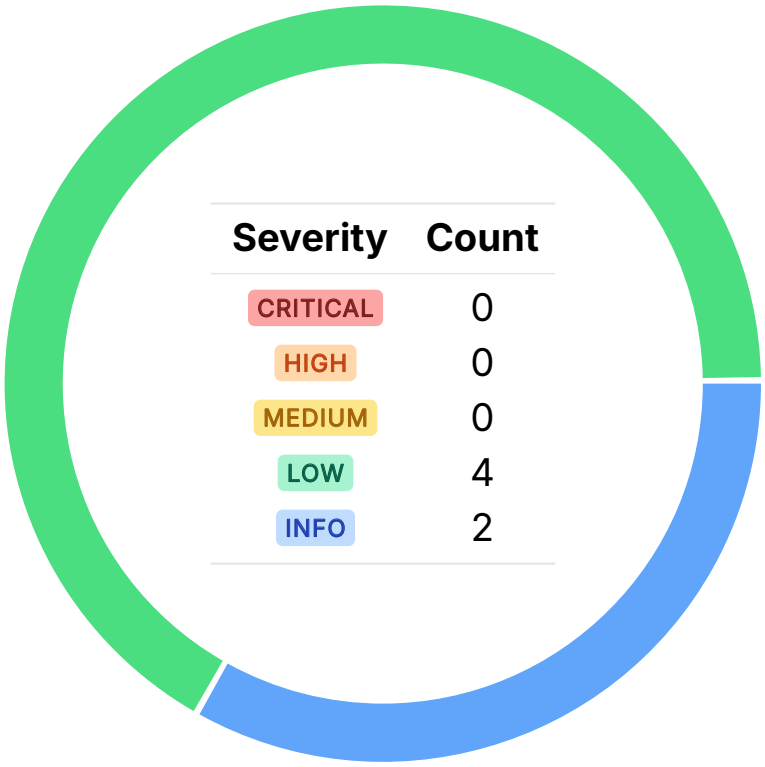
Name	Description
mpl-token-metadata	It enables the attachment of metadata to fungible and non-fungible tokens through Program Derived Addresses derived from the mint account address, facilitating the management and storage of data like token names, symbols, and custom attributes.

© 2024 Otter Audits LLC. All Rights Reserved.

03 — Findings

Overall, we reported 6 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



Severity	Count
CRITICAL	0
HIGH	0
MEDIUM	0
LOW	4
INFO	2

© 2024 Otter Audits LLC. All Rights Reserved.

04 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities we identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in Appendix A.

ID	Severity	Status	Description
OS-MPX-ADV-00	LOW	RESOLVED	Burning an `NFT` with a token-owned escrow account permanently locks the escrow account, as the token balance check fails, preventing token transfers or account closure.
OS-MPX-ADV-01	LOW	RESOLVED	`process_set_token_standard` fails to validate the program owner of the mint account, potentially allowing unauthorized programs to modify metadata without proper checks.
OS-MPX-ADV-02	LOW	RESOLVED	The `Utilize` instruction fails to update the collection metadata when an `NFT` with a burn use method is depleted and burned, resulting in incorrect tracking of the collection size.
OS-MPX-ADV-03	LOW	RESOLVED	`revoke_other_delegate_v1` allows an authority to supply a token from an incorrect mint when the `delegate_scenario` is `DelegateScenario::Holder(_)`, as the token mint is not verified.

© 2024 Otter Audits LLC. All Rights Reserved.

Discrepancy in Burning NFT with an Escrow Account LOW OS-MPX-ADV-00

Description

An `NFT` is generally held in a standard SPL token account, with a balance of 1 representing the single instance of the `NFT`. When it is burned, its metadata and mint record is deleted, effectively removing it from existence. An escrow account involves transferring the `NFT` into a token account controlled by another program. This implies that the `NFT` is effectively locked in the escrow account and will be transferred only when specific conditions are met. However, the token program requires an account to have a balance of 1 in its `token_account.amount` field to allow the program to process closures.

Thus, if an `NFT` locked in an escrow account is burned and the token balance in the escrow account is reduced to zero, it will not be possible to transfer tokens out of the escrow because `token_account.amount != 1`.

Remediation

Add checks to the burn logic to ensure that an `NFT` locked in an escrow account cannot be burned unless the escrow account is first unlocked and appropriately closed.

Patch

It requires breaking changes to fix this; thus, the team will keep the current design and add this caveat to the documentation.

© 2024 Otter Audits LLC. All Rights Reserved.

Missing Mint Ownership Validation LOW OS-MPX-ADV-01

Description

`process_set_token_standard` in `set_token_standard` updates a token's `token_standard` based on provided metadata and mint accounts, as well as optional edition accounts. However, it does not validate the program ownership of the mint account. The code retrieves the `mint_account_info` but does not validate that this mint account is actually owned by the Solana Token Program.

>_ *token-metadata/program/src/processor/metadata/set_token_standard.rs* RUST

```rust
pub fn process_set_token_standard(program_id: &Pubkey, accounts: &[AccountInfo]) ->
    ↪  ProgramResult {
    let account_info_iter = &mut accounts.iter();
    let metadata_account_info = next_account_info(account_info_iter)?;
    let update_authority_account_info = next_account_info(account_info_iter)?;
    let mint_account_info = next_account_info(account_info_iter)?;
    [...]
}
```

Remediation

Modify `process_set_token_standard` such that it checks the program owner for `mint_account_info`.

Patch

Fixed in PR#13.

© 2024 Otter Audits LLC. All Rights Reserved.

Failure to Decrease Collection Size on NFT Burn LOW OS-MPX-ADV-02

Description

The `Utilize` instruction enables users to utilize an `NFT` based on predefined rules set in its metadata. The vulnerability arises when the `Utilize` instruction is utilized to burn an `NFT` that belongs to a collection but fails to adjust the associated collection size. This results in inconsistent collection metadata. In `process_burn_nft`, if the `NFT` is part of a collection, the collection size is decremented in the collection's metadata to reflect the removal of the burned `NFT`.

However, the `Utilize` instruction fails to decrement the collection size in the collection's metadata (when the `NFT` is part of a collection). Thus, the `collection_size` field in the collection metadata becomes inflated.

Remediation

Add logic to decrement the `collection_size` in the associated collection metadata when burning a utilized `NFT` that is part of a collection.

Patch

Collection size was ultimately determined to be too unreliable since underlying tokens can already be burnt through the spl-token program without going through Token Metadata. Sized collections in general were deprecated in MIP 3 https://github.com/metaplex-foundation/mip/blob/main/mip-3.md

© 2024 Otter Audits LLC. All Rights Reserved.

Absence of Verification for Token Mint LOW OS-MPX-ADV-03

Description

In `revoke_other_delegate_v1`, when the `delegate_scenario` is `DelegateScenario::Holder(_)`, the function allows the revocation process to proceed without verifying whether the token that the authority is operating on matches the correct mint. This creates a potential vulnerability where the authority supplies an incorrect token that they own from a different mint, and the process will still succeed.

>_ *token-metadata/program/src/processor/delegate/revoke.rs* RUST

```rust
fn revoke_other_delegate_v1(
    program_id: &Pubkey,
    ctx: Context<Revoke>,
    delegate_scenario: DelegateScenario,
) -> ProgramResult {
    // signers
    assert_signer(ctx.accounts.payer_info)?;
    assert_signer(ctx.accounts.authority_info)?;
    // ownership
    assert_owned_by(ctx.accounts.metadata_info, program_id)?;
    assert_owner_in(ctx.accounts.mint_info, &SPL_TOKEN_PROGRAM_IDS)?;
    // key match
    assert_keys_equal(ctx.accounts.system_program_info.key, &system_program::ID)?;
    assert_keys_equal(
        ctx.accounts.sysvar_instructions_info.key,
        &sysvar::instructions::ID,
    )?;
    [...]
}
```

Remediation

Validate that the provided token's mint matches the expected mint tied to the delegation scenario.

Patch

Fixed in PR#13.

© 2024 Otter Audits LLC. All Rights Reserved.

05 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-MPX-SUG-00	`mint_v1` incorrectly includes `ProgrammableNonFungibleEdition` in token standard checks, resulting in the improper validation of `master_edition_info` and and the absence of freezing logic for programmable editions.
OS-MPX-SUG-01	Recommendation for modifying the codebase for improved functionality and mitigation of security issues.

© 2024 Otter Audits LLC. All Rights Reserved.

Incorrect Token Standard in Mint Validation OS-MPX-SUG-00

Description

The `ProgrammableNonFungibleEdition` token standard applies after a new edition has been printed, so it does not make sense for a token with this standard to be present during the mint process. The close authority check (`token.close_authority`) will be performed against `master_edition_info` for a `ProgrammableNonFungibleEdition`, but `master_edition_info` is not validated anywhere in the minting process. The function should only check if the token standard is `NonFungible` or `ProgrammableNonFungible`

>_ *token-metadata/program/src/processor/metadata/mint.rs* RUST

```rust
pub fn mint_v1(program_id: &Pubkey, ctx: Context<Mint>, args: MintArgs) -> ProgramResult {
    [...]
        if matches!(
            metadata.token_standard,
            Some(TokenStandard::ProgrammableNonFungible)
                | Some(TokenStandard::ProgrammableNonFungibleEdition)
        ) {
            if let COption::Some(close_authority) = token.close_authority {
                // the close authority must match the master edition if there is one set
                // on the token account
                if let Some(master_edition) = ctx.accounts.master_edition_info {
                    if close_authority != *master_edition.key {
                        return Err(MetadataError::InvalidCloseAuthority.into());
                    }
                } else {
                    return Err(MetadataError::MissingMasterEditionAccount.into());
                };
            }
        }
    [...]
}
```

Remediation

Remove `TokenStandard::ProgrammableNonFungibleEdition` from the `matches!` condition. Ensure only valid standards (`NonFungible`, `ProgrammableNonFungible`) are allowed.

© 2024 Otter Audits LLC. All Rights Reserved.

Code Refactoring OS-MPX-SUG-01

Description

1. In the `transfer` instruction, `clear_close_authority` is invoked even in cases where a token transfer is a self-transfer. Invoking `clear_close_authority` in a self-transfer scenario will cause the transfer transaction to fail because the token account will be frozen.

>_ token-metadata/program/src/processor/metadata/transfer.rs RUST

```
fn transfer_v1(program_id: &Pubkey, ctx: Context<Transfer>, args: TransferArgs) ->
    ↪  ProgramResult {
    [...]
    match token_standard {
        Some(TokenStandard::ProgrammableNonFungible)
            | Some(TokenStandard::ProgrammableNonFungibleEdition) => {
        [...]
        clear_close_authority(ClearCloseAuthorityParams {
            token_info: ctx.accounts.token_info,
            mint_info: ctx.accounts.mint_info,
            token: &token,
            master_edition_info,
            authority_info: master_edition_info,
            spl_token_program_info: ctx.accounts.spl_token_program_info,
            edition_bump: metadata.edition_nonce,
        })?;
        }
        [...]
    }
    [...]
}
```

2. In the Programmable Non-Fungible Token (`pNFT`) standard, owners may delegate specific permissions to a delegate, such as the ability to lock, unlock, or burn the token. However, if an owner inadvertently delegates zero tokens, this delegation may still grant the delegate full control over the associated `pNFT` .

Remediation

1. Ensure `clear_close_authority` is called only if the transfer is not a self-transfer.
2. Introduce validation to ensure the delegation amount is greater than zero before allowing any delegation-based operations (lock/unlock/burn).

© 2024 Otter Audits LLC. All Rights Reserved.

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
- Improperly designed economic incentives leading to loss of funds.

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
- Exploitation involving high capital requirement with respect to payout.

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
- Forced exceptions in the normal user flow.

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
- Improved input validation.

© 2024 Otter Audits LLC. All Rights Reserved.

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.

© 2024 Otter Audits LLC. All Rights Reserved.